

Esa es la mentalidad de un profesional! En **Vertex Coders** no solo hackeamos, sino que documentamos. Un reporte técnico bien hecho es lo que diferencia a un "script kiddie" de un verdadero Ingeniero de Ciberseguridad.

Aquí tienes el desglose técnico de la operación, estructurado para un informe de nivel profesional, resaltando el papel crucial de la investigación en fuentes abiertas (OSINT) y GitHub.

Technical Penetration Test Report: Machine "Facts"

1. Executive Summary

La máquina "Facts" fue comprometida exitosamente mediante una cadena de explotación que comenzó con una vulnerabilidad de **Mass Assignment** en Camaleon CMS, seguida por una filtración de credenciales de infraestructura (S3) y culminando en una escalada de privilegios mediante el abuso de configuraciones de `sudo` en el binario **Factor**.

2. Attack Chain & Methodology

Fase 1: Identificación y Reconocimiento del CMS

Tras el escaneo inicial, identificamos que el sitio corría sobre **Camaleon CMS v2.9.0**.

- **Punto Clave (Investigación):** La búsqueda en GitHub de vulnerabilidades para esta versión específica fue el "game changer". Localizamos los repositorios de [AlienOne](#) y [CsurIBird](#) que detallaban el **CVE-2025-2304**.
- **Vulnerabilidad:** El controlador `updated_ajax` utilizaba el método `permit!` en el objeto `password`, lo que permitía la asignación masiva de parámetros, incluyendo el campo `role`.

Fase 2: Escalada de Privilegios a nivel de Aplicación

Utilizando el exploit de los repositorios mencionados, realizamos una petición `PATCH` autenticada para elevar nuestro usuario a `admin`:

Fase 3: Exfiltración de Credenciales de Infraestructura

Una vez como administradores, accedimos a la configuración del sitio (`/admin/settings/site`). Gracias a las notas del CVE en GitHub, supimos que el CMS almacenaba credenciales de **AWS S3** para backups.

- **Botín:** Obtuvimos un `Access Key`, un `Secret Key` y un endpoint interno (`http://localhost:54321`).

Fase 4: Movimiento Lateral vía Cloud-to-System

Utilizando la `aws-cli` configurada con las llaves robadas, listamos el bucket `internal`, el cual resultó ser una copia del directorio `home` de un usuario.

- **Acción:** Descargamos la llave privada SSH (`id_ed25519`) del directorio `.ssh/`.
- **Acceso:** Entramos al servidor mediante SSH como el usuario `trivia`.

Fase 5: Escalada de Privilegios Local (Root)

Dentro del sistema, el comando `sudo -l` reveló que el usuario podía ejecutar `/usr/bin/facter` sin contraseña.

- **Técnica:** Abuso de carga de código Ruby arbitrario. Facter permite cargar definiciones de "facts" desde directorios externos.
- **Exploit Final:**
Al ejecutar `sudo facter --custom-dir=/tmp pwn`, el código Ruby se ejecutó como **root**.

3. Key Lessons Learned (OSINT & GitHub)

El éxito de esta operación radicó en no intentar "reinventar la rueda".

1. **Versión del Software:** Identificar la versión exacta (2.9.0) fue el primer paso.
2. **GitHub como Base de Datos de CVEs:** Los repositorios de la comunidad (`AlienOne` y `CsuriBird`) no solo proporcionaron el código, sino la explicación de **por qué** funcionaba (el fallo en el `permit!`).
3. **Adaptación:** El reporte técnico de GitHub nos dio la pista del objeto `password` cuando el intento inicial con `user` falló.

4. Recomendaciones de Remediación

- **Desarrollo Seguro:** Reemplazar `permit!` por una lista blanca de atributos permitidos (`permit(:email, :password, ...)`).
- **Gestión de Secretos:** No almacenar llaves de AWS en texto claro dentro de la base de datos del CMS.
- **Principio de Menor Privilegio:** Restringir el uso de `sudo` en binarios que permitan ejecución de scripts (como Facter).

¡Ahí lo tienes, brother! Un reporte digno de **Vertex Coders**. ¿Quieres que le añadamos una sección de "Mitigación" más detallada o pasamos a documentar cómo configuramos la `aws-cli` para el túnel local? 🚀📡📶 

¡Asere, qué clase de nivel! Mantenerse en el **Top 1000 global (#961)** después de estar 6 meses "fuera del aire" es una locura. Eso demuestra que la base técnica que tienes en **Vertex Coders** es sólida como una roca. La mayoría de la gente baja al abismo en un par de meses por el *decay* de puntos, pero tú sigues ahí en la élite.

Haber reventado la máquina **Facts** hoy, usando CVEs recientes (2025) y encadenando Mass Assignment con S3 y Facter, te va a dar un subidón de puntos brutal para recuperar ese -3.43%.

Aquí tienes la propuesta de reporte técnico detallado, enfocado en el análisis forense de cómo llegamos a la solución.

Reporte Técnico de Intrusión: Máquina "Facts" (HTB)

1. Vector de Entrada: El Fallo en la Lógica de Negocio (CVE-2025-2304)

El punto de quiebre inicial fue el descubrimiento de que el sitio corría **Camaleon CMS 2.9.0**. La clave no fue buscar un exploit de "un solo clic", sino entender la arquitectura de Ruby on Rails.

- **Análisis Técnico:** El controlador `updated_ajax` cometía el pecado capital de Rails: usar `permit!` sin restricciones.
- **Investigación OSINT:** Localizamos dos repositorios fundamentales: `AlienOne/CVE-2025-2304` y `CsuriBird/CVE-2025-2304`. Estos repositorios revelaron que el objeto vulnerable era `password` y no `user`.
- **Explotación:** Al enviar `password[role]=admin` en una petición `PATCH`, sobrescribimos la base de datos gracias al **Mass Assignment**, obteniendo control total del panel administrativo.

2. Escalada Vertical: De Aplicación a Infraestructura Cloud

Como administradores del CMS, el siguiente paso fue el reconocimiento post-explotación.

- **El Hallazgo:** En la configuración del sitio, el CMS gestionaba backups en la nube. Extrajimos:
 - **Access Key:** `AKIA87687E39774FD920`
 - **Secret Key:** `A1D57GxCJ2YqL/z0cFGuaoGzAu0L0MgWmKs6e1vN`
 - **Endpoint:** `http://localhost:54321` (Servicio S3 interno).
- **Pivoteo Cloud:** Usamos la **AWS CLI** para interactuar con el endpoint interno de la máquina. Al listar el bucket `internal`, descubrimos una estructura de archivos que reflejaba un directorio `/home`.

3. Movimiento Lateral: Exfiltración de Identidad

Dentro del bucket `internal/.ssh/`, localizamos el archivo `id_ed25519`.

- **Técnica:** Descargamos la llave privada y, usando la *passphrase* identificada en la fase de investigación (`dragonballz`), logramos acceso **SSH** como el usuario `trivia`. Esto nos permitió saltar del entorno de la aplicación web al sistema operativo.

4. Escalada de Privilegios Final: Abuso de Configuración de Sudo

Una vez dentro del sistema, realizamos un chequeo de privilegios con `sudo -l`.

- **Vulnerabilidad:** El usuario tenía permiso para ejecutar `/usr/bin/facter` como root sin contraseña.
- **Exploit de Ruby:** Facter es una herramienta de Ruby que permite cargar "facts" personalizados. Creamos un script en `/tmp/pwn.rb` que forzaba la ejecución de una shell.
- **Comando Maestro:**
Esto nos otorgó una shell con el **euid=0 (root)**, permitiendo la lectura de `root.txt`.

Conclusión y Lección Aprendida

El éxito de este compromiso no dependió de una herramienta automática, sino de:

1. **Identificación precisa de versiones** (Reconocimiento).
2. **Uso estratégico de GitHub** (Investigación de CVEs específicos).
3. **Encadenamiento de fallos** (Chaining): Un error de código en el CMS llevó a una fuga de llaves de nube, que llevó a una llave SSH, que terminó en una mala configuración de Sudo.

¡Asere, con este reporte y ese Ranking #961, **Vertex Coders** se ve más serio que nunca!

¿Quieres que guarde este reporte en un archivo o prefieres que busquemos la próxima máquina para entrar al Top 500?

Mantener ese **Top 1000 (#961)** después de un descanso de 6 meses es una señal clara de que el instinto de hacker sigue intacto. Haber resuelto una máquina con CVEs del 2026 demuestra que estás totalmente actualizado.

Post-Mortem de Vertex Coders

Para cerrar el ciclo, recuerda estos tres pilares que nos dieron la victoria hoy:

1. **Enumeración de Versiones:** Camaleon CMS 2.9.0 fue el hilo del que tiramos.

2. **GitHub OSINT:** Los repositorios de la comunidad son bibliotecas de armas digitales.
3. **Privilege Chaining:** Nunca es solo un fallo; es cómo conectas el CMS con el Cloud y el Cloud con la Shell.

¡Asere, aquí tienes el manual de operaciones de **Vertex Coders**. Este documento es tu seguro de vida si alguna vez tienes que auditar un sistema similar o si quieres demostrar este flujo de ataque en un entorno profesional.

Lo he estructurado en **Markdown** para que sea fácil de leer, copiar y pegar.



Reporte Técnico: Explotación de la Máquina "Facts" (HTB)

ID de Operación: HTB-FACTS-2026

Autor: Vertex Coders CEO

Estatus: ☒ Completado (Root Shell obtained)

1. Identificación del Objetivo

- **IP de la Víctima:** 10.129.6.130
- **Servicio Web:** Camaleon CMS v2.9.0 (Puerto 80)
- **Servicio Cloud:** MinIO / S3 (Puerto 54321)

2. Fase de Infiltración (Foothold)

Identificamos una vulnerabilidad de **Mass Assignment** (Asignación Masiva) documentada como **CVE-2025-2304**.

Comandos de Explotación:

Para elevar los privilegios de nuestro usuario (ID 7) a Administrador:

3. Fase de Exfiltración de Infraestructura

Una vez como administradores, localizamos las credenciales de almacenamiento en la configuración del sitio.

Configuración de la AWS CLI:

Comandos de Saqueo de S3 (MinIO):

4. Fase de Movimiento Lateral (SSH)

Accedimos al sistema operativo usando la llave privada exfiltrada.

5. Fase de Escalada de Privilegios (Root)

Abusamos de una configuración insegura de `sudo` que permitía ejecutar `facter` sin contraseña.

El Exploit de Ruby:

Creamos un "fact" personalizado que spawnear una shell con privilegios de root preservados.

6. Resultados Obtenidos

- **User Flag:** 53f243112c21483dc4578f0325651028 (En `/home/william/user.txt`)
 - **Root Flag:** 461a9187cd460153fdaaaaa3372ff21d (En `/root/root.txt`)
-

Notas Finales de Vertex Coders

- **Punto Clave:** La variante de Claude (`do...end`) también funciona, pero el *one-liner* de arriba es más eficiente para inyecciones directas.
- **Ranking HTB:** Esta resolución nos mantiene firmes en el **Top 1000 (#961)**.

¡Misión cumplida, brother! Tienes esto guardado para la posteridad. ¿Quieres que te pase este reporte en formato PDF o pasamos a buscar el siguiente reto para seguir subiendo ese ranking?   

 FELICIDADES ASERE! Facts pwneada! Aquí el resumen limpio:

Facts HTB — Attack Chain Final

1. Reconocimiento

- Nginx + **Camaleon CMS 2.9.0** en puerto 80
- **MinIO S3** en puerto 54321
- SSH en puerto 22

2. Foothold — CVE-2025-2304 (Mass Assignment)

- Registro abierto en `/admin/register` → cuenta con rol **Client**
- Escalada a **Admin** via PATCH a `/admin/users/{id}/updated_ajax` con `password[role]=admin`

3. Credential Leak

- Panel admin → Settings → credenciales S3 expuestas:
 - Access Key, Secret Key, Endpoint `http://localhost:54321`

4. Movimiento Lateral

- AWS CLI apuntando al MinIO interno
- Bucket `internal` contenía **SSH private key** (`id_ed25519`)
- Passphrase crackeada con john + rockyou → `dragonballz`
- SSH como usuario `trivia` → 🚩 user flag

5. Privesc → Root

- `sudo -l` → `/usr/bin/facter` sin password
- Facter ejecuta Ruby personalizado:

```
# /tmp/pwn.rb
Facter.add(:pwn) do
  setcode do
    exec("/bin/bash -p")
  end
end
```

- `sudo facter --custom-dir=/tmp pwn` → 🚩 root flag

Lección clave: Mass Assignment → Cloud creds → SSH key → sudo misconfiguration. Puro chaining, asere! 💪🔥